

HelixTranslate

HelixTranslate

Traducción de libros con modelo verificado — honesta por diseño, nunca un respaldo silencioso.

Resumen

HelixTranslate es una plataforma de traducción de libros electrónicos de alto rendimiento basada en Go, que traduce obras entre más de 100 idiomas utilizando proveedores de LLM verificados, con monitoreo en tiempo real de WebSocket y una política estricta de *no respaldo silencioso* que falla de manera explícita en lugar de degradarse sin aviso.

Descripción breve

Kit de herramientas de traducción universal de libros electrónicos basado en Go. Traduce formatos FB2, EPUB, TXT, HTML, PDF y DOCX entre más de 100 idiomas utilizando los modelos LLM verificados más robustos (a través del puente LLMsVerifier), con APIs REST/HTTP-3 y gRPC, procesamiento distribuido y un panel de monitoreo en tiempo real de WebSocket.

Descripción detallada

HelixTranslate es un sistema de grado empresarial basado en Go para traducir libros completos entre idiomas, utilizando proveedores de LLM —no párrafos ni fragmentos, sino obras de principio a fin—. Analiza y regenera múltiples formatos de libros electrónicos (FB2, EPUB, TXT, HTML, PDF, DOCX), admite más de 100 idiomas con detección automática y ofrece tanto herramientas CLI como servidores API (REST sobre HTTP/3, gRPC y un flujo de eventos WebSocket), lo que le permite integrarse tanto en flujos de trabajo de terminal como en mallas de servicios. Su característica definitoria es *cómo elige un modelo*: en lugar de codificar un proveedor y esperar que siga

funcionando, HelixTranslate delega toda la autoridad del modelo al puente LLMsVerifier (pkg/bridge), que selecciona el modelo API verificado más potente y devuelve una cadena de respaldo determinista ordenada por puntuación. La elegibilidad del modelo se decide mediante una puntuación ponderada que evalúa capacidad de respuesta, código, riqueza de funciones y fiabilidad —así, el modelo que realiza la traducción gana su lugar demostrando que funciona, no por aparecer en un archivo de configuración—.

De manera crucial, el sistema aplica un mandato de “*no respaldo silencioso*” directamente en el código: si no hay presente una clave de proveedor API, o un operador solicita explícitamente un proveedor no disponible, el proceso devuelve un error forzado honesto en lugar de cambiar de proveedor discretamente o recurrir a un entorno local fingiendo que todo está bien —una regla reforzada por una compuerta previa a la compilación y una prueba de mutación asociada—. Los entornos locales (Ollama, llama.cpp) fueron eliminados deliberadamente de la ruta predeterminada para que un motor más débil nunca pueda sustituir silenciosamente a uno verificado. Alrededor del núcleo de traducción opera un subsistema de monitoreo en tiempo real de WebSocket: la herramienta de traducción CLI emite eventos tipados a un servidor de monitoreo que alimenta un panel web en vivo, mientras que los trabajadores remotos SSH distribuyen la carga de trabajo para la traducción paralela. Sobre esta base se superponen pasadas de pulido para garantizar coherencia, un análisis de calidad en fase de preparación, almacenamiento en caché de traducciones para controlar costos en textos extensos y un sistema de control de calidad basado en visión. Toda la plataforma responde a una constitución de ingeniería anti-engaño: las pruebas deben demostrar resultados reales y visibles para el usuario, respaldadas por pruebas de mutación obligatorias en lugar de marcas verdes que no prueban nada.

Por qué lo creamos

Para traducir libros de formato largo de manera fiable y *honesta*, sin entregar nunca una traducción “degradada pero presente”. El principio de diseño es que una traducción ausente o no verificable debe ser un error claro y contundente, y que la selección del modelo debe resolverse siempre en un proveedor genuinamente verificado, en lugar de basarse en una suposición predefinida o un respaldo local silencioso.

Por qué es un cambio radical

La mayoría de los flujos de traducción LLM fallan en silencio: recurren discretamente a un modelo más débil, retroceden a un entorno local o emiten una salida parcial mientras el conjunto de pruebas sigue mostrando resultados positivos y nadie detecta el abismo de calidad. HelixTranslate elimina por completo esa modalidad de fallo: la elección del modelo está sujeta a verificación, la cadena de respaldo es determinista y totalmente transparente, y la ausencia de clave o modelo verificado se resuelve en un error contundente, no en un encogimiento de hombros silencioso. Esa única decisión de diseño convierte la pregunta “¿Esta traducción se ejecutó realmente en un modelo capaz y verificado?” de una esperanza imposible de comprobar en una garantía que el sistema impone en tu nombre.

Qué hay de innovador

- **Enrutamiento de modelos con verificación** mediante el puente LLMsVerifier: se selecciona automáticamente el modelo *verificado* más potente, de modo que los operadores declaran su intención, no nombres de proveedores, y nunca eligen manualmente un proveedor que podría estar inactivo.
- **Garantía de no respaldo silencioso aplicada en código:** cuatro rutas de enrutamiento explícitas (simulación / verificador explícito / proveedor explícito / puente por defecto), cada una de las cuales genera un error contundente en lugar de cambiar de forma silenciosa, además de la eliminación deliberada de entornos locales en la ruta predeterminada para que no haya nada más débil *a lo que recurrir*.
- **Aplicación mecánica:** una compuerta previa a la compilación CM-NO-LOCAL-RUNTIME junto con una prueba de mutación emparejada que, en tiempo de compilación, verifica que nunca se construya un cliente de entorno local en la ruta predeterminada. La garantía no puede degradarse porque la compilación falla si ocurre.
- **Cadena de respaldo determinista y ordenada por puntuación:** se permite el failover entre proveedores de modelos *verificados* y es totalmente transparente, una distinción fundamentada respecto al respaldo silencioso prohibido. Siempre sabes qué modelo capaz asumió el trabajo.
- **Monitoreo WebSocket en tiempo real:** eventos de traducción tipados transmitidos en vivo a un panel de control, con trabajadores SSH distribuidos para que un trabajo de longitud de libro sea visible y paralelo, no una caja negra.
- **Régimen de pruebas anti-engaño:** pruebas de mutación, aserciones negativas, ejecuciones en sistemas reales y control de calidad basado en visión que, en conjunto, garantizan que

*“las pruebas pasan” nunca pueda enmascarar silenciosamente
“la función no funciona en realidad”.*

Principales desafíos técnicos y cómo los resolvimos

- **Garantizar un flujo de traducción honesto (sin degradación silenciosa).** Se resolvió centralizando toda la autoridad de los modelos en el puente LLMsVerifier para contar con un único punto de decisión que supervisar, codificando cuatro ramas de enrutamiento explícitas que fallan de manera contundente en lugar de adivinar, eliminando por completo los respaldos de entorno local de la ruta predeterminada y soldando la regla con una compuerta de compilación más una prueba de mutación que hace fallar la compilación si se elimina la garantía.
- **“Pruebas verdes, funciones rotas”.** La constitución nombra explícitamente este modo de fallo y lo neutraliza con el régimen de Pruebas Anti-Engaño: aserciones concretas y visibles para el usuario en lugar de detalles de implementación, sistemas reales en el ciclo (los simulacros se limitan a pruebas unitarias), pruebas de mutación obligatorias (romper deliberadamente la función *debe* hacer que la prueba falle) y control de calidad verificado por visión que realmente examina la salida.
- **Calidad en formatos largos y múltiples formatos.** Las entradas del tamaño de un libro ponen a prueba tanto la coherencia como el presupuesto. Se resolvió con un pulido en múltiples pasadas que revisita el texto, un análisis en fase de preparación que dimensiona el trabajo de antemano y una caché de traducciones que evita pagar dos veces por el mismo pasaje.

Pila tecnológica

- **Go** — elegido por sus primitivas de concurrencia, que se adaptan de forma natural al análisis, traducción y transmisión simultánea de múltiples capítulos; el backend de alta concurrencia, módulo `digital.vasic.translator`.
- **Gin** — seleccionado como enrutador HTTP rápido y minimalista para servir la superficie REST API.
- **QUIC / HTTP/3 (quic-go)** — elegidos para proporcionar a la superficie REST API un transporte moderno de baja latencia que resiste en redes imperfectas.
- **gRPC + Protocol Buffers** — escogidos para una interfaz de servicio fuertemente tipada y de alto rendimiento que opera junto a REST para usuarios programáticos.

- **Gorilla WebSocket** — utilizado para transportar el flujo de eventos de traducción en tiempo real y tipado que alimenta el panel de monitoreo en vivo.
- **PostgreSQL, SQLite, Redis** — división deliberada en tres capas: PostgreSQL para datos relacionales persistentes, SQLite para el estado local/incrustado (también respalda el almacén de modelos verificados del puente, `data/verified_models.db`) y Redis como caché activa.
- **unidoc/unioffice + unipdf** — seleccionados para manejar los formatos complejos: análisis y regeneración de DOCX y PDF, garantizando que los libros electrónicos multiformato se mantengan fieles en el proceso de ida y vuelta.
- **Cobra** — elegido como marco de trabajo para la CLI que impulsa `unified-translator` y sus herramientas complementarias.
- **golang-jwt (JWT HS256)** — adoptado para la autenticación API sin estado, combinado con un sistema de limitación de tasa por token por IP y seguridad en el transporte TLS/QUIC para reforzar la superficie.
- **Puente LLMsVerifier (pkg/bridge)** — el eje central: obtiene el modelo verificado más robusto junto con su cadena de respaldo determinista, y actúa como único punto de aplicación de la garantía de *no-respaldo-silencioso*.
- **Testify** — elegido para la suite de pruebas de Go, que incluye el archivo dedicado `provider_routing_test.go` y las compuertas de mutación que mantienen íntegras las reglas de honestidad.
- **Docker / Podman (sin root) + Compose** — seleccionados para el despliegue contenerizado y distribuido (`docker-compose.distributed.yml`), con Podman sin root para un perfil de seguridad más estricto.

Estado y notas sobre transparencia

- **Estado: beta.** Plataforma funcional; la versión se indica de manera inconsistente en `VERSION/Makefile/AGENTS.md`, por lo que se considera no consolidada.
- **Licencia: por definir.** El archivo `README` menciona MIT, pero esto no se ha confirmado con un archivo `LICENSE` — verificar antes de afirmarlo.
- Los puntos finales del panel de control/monitoreo son solo para `localhost` y no son públicos. Las cifras de rendimiento de WebSocket en la documentación son objetivos declarados, no verificados. `ARCHITECTURE.md` aún incluye motores Ollama/locales eliminados (información desactualizada).

Nivel de prioridad: Helix-principal (clúster LLM-infraestructura). Se ubica dentro de la familia de plataformas Helix, después de HelixTrack.